

Week 11: Splines

27/03/26

Overview

In this lab we'll be fitting a second-order P-Splines regression model to foster care entries by state in the US, projecting out to 2030.

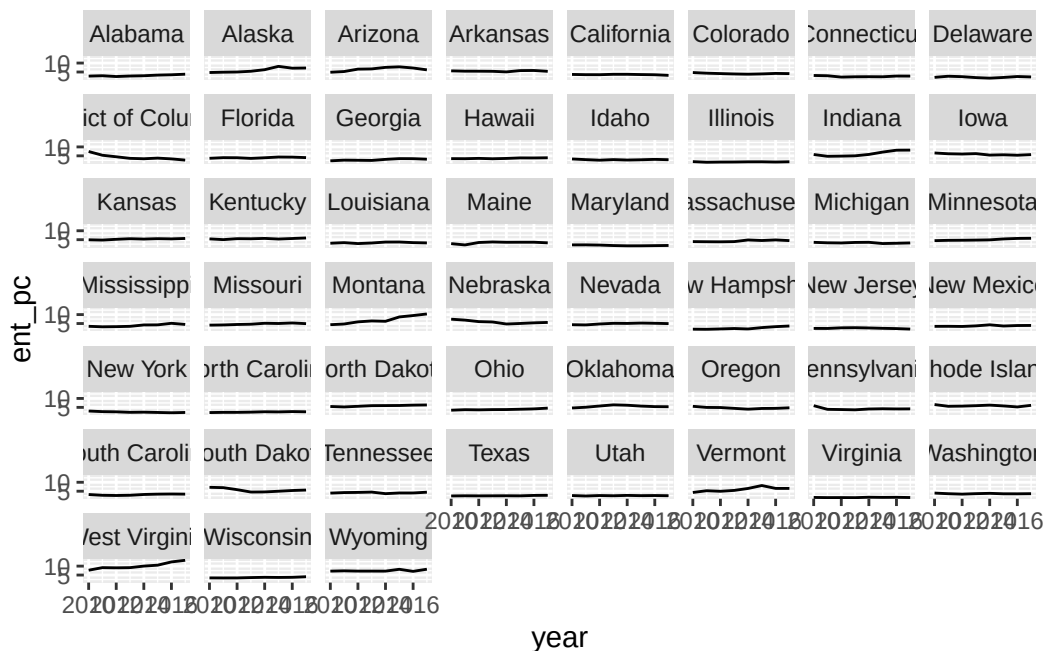
```
library(tidyverse)
library(here)
library(rstan)
library(tidybayes)
source(here("code/getsplines.R"))
```

Here's the data

```
d <- read_csv(here("data/fc_entries.csv"))
```

Let's make a plot highlighting trends over time by state.

```
d |>
  ggplot(aes(year, ent_pc)) +
  geom_line()+
  facet_wrap(~state)
```



P-Splines

Let's fit a hierarchical second-order P-Splines regression model to estimate the (logged) entries per capita over the period 2010-2017. The model we are fitting is

$$\begin{aligned}
 y_{st} &\sim N(\log \lambda_{st}, \sigma_{y,s}^2) \\
 \log \lambda_{st} &= \alpha_k B_k(t) \\
 \Delta^2 \alpha_k &\sim N(0, \sigma_{\alpha,s}^2) \\
 \log \sigma_{\alpha,s} &\sim N(\mu_\sigma, \tau^2)
 \end{aligned}$$

Where $y_{s,t}$ is the logged entries per capita for state s in year t . We use cubic splines that have knots 2.5 years apart and are a constant shape at the boundaries. Standard normal priors are on standard deviations and hyperparameters.

Get the data in the right format:

```

years <- unique(d$year)
states <- unique(d$state)
N <- length(years)
S <- length(unique(d$state))
y <- d |>
  select(state, year, ent_pc) |>
  mutate(log_ent = log(ent_pc)) |>

```

```

select(-ent_pc) |>
pivot_wider(names_from = "state", values_from = "log_ent") |>
select(-year) |>
as.matrix()

pop <- d %>%
  dplyr::select(state, year, child_acs) %>%
  pivot_wider(names_from = "state", values_from = "child_acs") %>%
  dplyr::select(-year) %>% as.matrix()

Bs <- getsplines(years, I = 2.5)$B.ik

stan_data <- list(y = y,
                  B = Bs,
                  K = ncol(Bs),
                  N = N,
                  S = S)

```

Run the model

```
mod <- stan(data = stan_data, file = "../code/models/fc_splines.stan")
```

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 1).

Chain 1:

Chain 1: Gradient evaluation took 0.000251 seconds

Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 2.51 seconds.

Chain 1: Adjust your expectations accordingly!

Chain 1:

Chain 1:

Chain 1: Iteration: 1 / 2000 [0%] (Warmup)

Chain 1: Iteration: 200 / 2000 [10%] (Warmup)

Chain 1: Iteration: 400 / 2000 [20%] (Warmup)

Chain 1: Iteration: 600 / 2000 [30%] (Warmup)

Chain 1: Iteration: 800 / 2000 [40%] (Warmup)

Chain 1: Iteration: 1000 / 2000 [50%] (Warmup)

Chain 1: Iteration: 1001 / 2000 [50%] (Sampling)

Chain 1: Iteration: 1200 / 2000 [60%] (Sampling)

Chain 1: Iteration: 1400 / 2000 [70%] (Sampling)

Chain 1: Iteration: 1600 / 2000 [80%] (Sampling)

Chain 1: Iteration: 1800 / 2000 [90%] (Sampling)
Chain 1: Iteration: 2000 / 2000 [100%] (Sampling)
Chain 1:
Chain 1: Elapsed Time: 16.018 seconds (Warm-up)
Chain 1: 11.714 seconds (Sampling)
Chain 1: 27.732 seconds (Total)
Chain 1:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 2).

Chain 2:
Chain 2: Gradient evaluation took 0.000344 seconds
Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 3.44 seconds.
Chain 2: Adjust your expectations accordingly!
Chain 2:
Chain 2:
Chain 2: Iteration: 1 / 2000 [0%] (Warmup)
Chain 2: Iteration: 200 / 2000 [10%] (Warmup)
Chain 2: Iteration: 400 / 2000 [20%] (Warmup)
Chain 2: Iteration: 600 / 2000 [30%] (Warmup)
Chain 2: Iteration: 800 / 2000 [40%] (Warmup)
Chain 2: Iteration: 1000 / 2000 [50%] (Warmup)
Chain 2: Iteration: 1001 / 2000 [50%] (Sampling)
Chain 2: Iteration: 1200 / 2000 [60%] (Sampling)
Chain 2: Iteration: 1400 / 2000 [70%] (Sampling)
Chain 2: Iteration: 1600 / 2000 [80%] (Sampling)
Chain 2: Iteration: 1800 / 2000 [90%] (Sampling)
Chain 2: Iteration: 2000 / 2000 [100%] (Sampling)
Chain 2:
Chain 2: Elapsed Time: 24.059 seconds (Warm-up)
Chain 2: 11.339 seconds (Sampling)
Chain 2: 35.398 seconds (Total)
Chain 2:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 3).

Chain 3:
Chain 3: Gradient evaluation took 0.000152 seconds
Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 1.52 seconds.
Chain 3: Adjust your expectations accordingly!
Chain 3:
Chain 3:
Chain 3: Iteration: 1 / 2000 [0%] (Warmup)
Chain 3: Iteration: 200 / 2000 [10%] (Warmup)
Chain 3: Iteration: 400 / 2000 [20%] (Warmup)

```

Chain 3: Iteration: 600 / 2000 [ 30%] (Warmup)
Chain 3: Iteration: 800 / 2000 [ 40%] (Warmup)
Chain 3: Iteration: 1000 / 2000 [ 50%] (Warmup)
Chain 3: Iteration: 1001 / 2000 [ 50%] (Sampling)
Chain 3: Iteration: 1200 / 2000 [ 60%] (Sampling)
Chain 3: Iteration: 1400 / 2000 [ 70%] (Sampling)
Chain 3: Iteration: 1600 / 2000 [ 80%] (Sampling)
Chain 3: Iteration: 1800 / 2000 [ 90%] (Sampling)
Chain 3: Iteration: 2000 / 2000 [100%] (Sampling)
Chain 3:
Chain 3: Elapsed Time: 15.442 seconds (Warm-up)
Chain 3:                10.232 seconds (Sampling)
Chain 3:                25.674 seconds (Total)
Chain 3:

```

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 4).

```

Chain 4:
Chain 4: Gradient evaluation took 0.000154 seconds
Chain 4: 1000 transitions using 10 leapfrog steps per transition would take 1.54 seconds.
Chain 4: Adjust your expectations accordingly!
Chain 4:
Chain 4:
Chain 4: Iteration: 1 / 2000 [ 0%] (Warmup)
Chain 4: Iteration: 200 / 2000 [ 10%] (Warmup)
Chain 4: Iteration: 400 / 2000 [ 20%] (Warmup)
Chain 4: Iteration: 600 / 2000 [ 30%] (Warmup)
Chain 4: Iteration: 800 / 2000 [ 40%] (Warmup)
Chain 4: Iteration: 1000 / 2000 [ 50%] (Warmup)
Chain 4: Iteration: 1001 / 2000 [ 50%] (Sampling)
Chain 4: Iteration: 1200 / 2000 [ 60%] (Sampling)
Chain 4: Iteration: 1400 / 2000 [ 70%] (Sampling)
Chain 4: Iteration: 1600 / 2000 [ 80%] (Sampling)
Chain 4: Iteration: 1800 / 2000 [ 90%] (Sampling)
Chain 4: Iteration: 2000 / 2000 [100%] (Sampling)
Chain 4:
Chain 4: Elapsed Time: 45.871 seconds (Warm-up)
Chain 4:                13.755 seconds (Sampling)
Chain 4:                59.626 seconds (Total)
Chain 4:

```

Future projection

Let's project forward entries per capita to 2030.

```
yhat <- mod %>%
  gather_draws(lambda[i,s]) %>%
  mutate(fit = exp(.value)) %>%
  median_qi() %>%
  mutate(year = years[i])

Bs_full <- getsplines(I = 2.5, 2000:2030)$B.ik

alphas <- extract(mod)[["alpha"]]
sigmas <- extract(mod)[["sigma"]]
sigma_ys <- extract(mod)[["sigma_y"]]

proj_years <- 2018:2030
K <- ncol(Bs)
K_full <- ncol(Bs_full)
proj_steps <- K_full - K

nsims <- nrow(alphas)

proj <- tibble()

alphas_proj <- array(NA, c(nsims, proj_steps, length(states)))

set.seed(1098)
# project the alphas
for(j in 1:length(states)){
  first_next_alpha <- rnorm(n = nsims,
                           mean = 2*alphas[,K,j] - alphas[,K-1,j],
                           sd = sigmas[,j])
  second_next_alpha <- rnorm(n = nsims,
                             mean = 2*first_next_alpha - alphas[,K,j],
                             sd = sigmas[,j])

  alphas_proj[,1,j] <- first_next_alpha
  alphas_proj[,2,j] <- second_next_alpha
  # now project the rest
  for(i in 3:proj_steps){
    alphas_proj[,i,j] <- rnorm(n = nsims,
                              mean = 2*alphas_proj[,i-1,j] - alphas_proj[,i-2,j],
```

```

sd = sigmas[,j])
}
}

# now use these to get y's
y_proj <- array(NA, c(nsim, length(proj_years), length(states)))
for(i in 1:length(proj_years)){
  for(j in 1:length(states)){
    all_alphas <- cbind(alphas[,j], alphas_proj[,j] )
    this_lambda <- all_alphas %*% as.matrix(Bs_full[length(years)+i, ])
    #y_proj[,i,j] <- rnorm(n = nsim, mean = this_lambda, sd = sigma_ys[,j])
    y_proj[,i,j] <- this_lambda
  }
}

proj_df <- tibble()

for(i in 1:length(proj_years)){
  for(j in 1:length(states)){
    proj_df<- bind_rows(proj_df, tibble(year = proj_years[i],
                                         state = states[j],
                                         median = exp(quantile(y_proj[,i,j], 0.5)),
                                         lower = exp(quantile(y_proj[,i,j], 0.65)),
                                         upper = exp(quantile(y_proj[,i,j], 0.35))))
  }
}

d %>%
  dplyr::select(state, year, ent_pc) %>%
  mutate(s = rep(1:length(states), each = nrow(y))) %>%
  left_join(yhat %>% ungroup() %>% dplyr::select(s,year, fit:fit.upper)) %>%
  filter(state %in% c("Texas", "California", "Mississippi", "Ohio")) %>%
  ggplot(aes(year, ent_pc)) +
  geom_point(aes(color = state)) +
  geom_line(aes(year, fit, color = state)) +
  geom_ribbon(aes(ymin = fit.lower, ymax = fit.upper, fill = state), alpha = 0.2) +
  geom_line(data = proj_df %>% filter(year < 2026, state %in% c("Texas", "California", "Mississippi", "Ohio"))) +
  geom_ribbon(data = proj_df %>% filter(year < 2026, state %in% c("Texas", "California", "Mississippi", "Ohio"))) +
  theme_bw() + ggtitle("Estimated and projected entries per capita \nsecond-order P-splines")

```

Estimated and projected entries per capita
second-order P-splines

